

EXPERIMENT – 8

Strings and Pattern matching

Aim: Explore key algorithms for string matching, including Rabin-Karp, Z-Algorithm, and Knuth-Morris-Pratt (KMP).

Objectives:

1. Build Strong Foundations in String matching algorithms
2. Solve complex problems using String and pattern matching

Lab outcome:

At the end of the lab session, students will be able to....

1. Build Strong Foundations in String matching algorithms
2. Solve complex problems using String and pattern matching

Prerequisite: Data structures, Analysis of algorithms, any programming language (C++/Java/Python)

CO Mapping:

CO4: Implement efficient pattern-matching algorithms, optimizing performance through detailed complexity analysis.

Po Mapping: PO1, PO2, PO3, PO4, PO5

Theoretical Description:

Lab Exercises:

1. Find the longest palindromic substring in a given string.
2. Implement the KMP pattern matching algorithm to search for a pattern in a text.
3. Count all palindromic substrings of a string efficiently.
4. Find all occurrences of a pattern in a text using the Rabin-Karp algorithm.
5. Find the smallest window in a string that contains all characters of another string.

Advanced Learner Challenges:

1. Compute the Longest Prefix which is also a Suffix (LPS Array) for a string.
2. Find the shortest string that contains all permutations of another string.
3. Generate all palindromic substrings of a string efficiently.
4. Implement the Z-Algorithm for pattern matching in a string.

Theory and Approaches:

1. Find the longest palindromic substring in a given string

Given a string *s*, return the longest *palindromic substring* in *s*.

Example 1:

Input: *s* = "babad"

Output: "bab"

Explanation: "aba" is also a valid answer.

Example 2:

Input: *s* = "cbabd"

Output: "bb"

Leetcode Link: 5: <https://leetcode.com/problems/longest-palindromic-substring/description/>

[Naive Approach] Generating all sub-strings – $O(n^3)$ time and $O(1)$ space

Generate all possible substrings of the given string. For each substring, check if it is a palindrome. If it is, update the result if its length is greater than the longest palindrome found so far.

[Better Approach – 2] Using Expansion from center – $O(n^2)$ time and $O(1)$ space

The idea is to traverse each character in the string and treat it as a potential center of a palindrome, trying to expand around it in both directions while checking if the expanded substring remains a palindrome.

=> For each position, we check for both odd-length palindromes (where the current character is the center) and even-length palindromes (where the current character and the next character together form the center).

=> As we expand outward from each center, we keep track of the start position and length of the longest palindrome found so far, updating these values whenever we find a longer valid palindrome.

Step-by-step approach:

- Iterate through each character in the string, treating it as the center of a potential palindrome.
- For each center, expand in two ways: one for odd-length palindromes (center at index *i*) and one for even-length palindromes (center between indices *i* and *i+1*)
- Use two pointers *low* and *high* to track the left and right boundaries of the current palindrome.
- While *low* and *high* are in bounds and *s*[*low*] == *s*[*high*], expand outward.
- If the current palindrome length (*high* – *low* + 1) is greater than the previous maximum, update the starting index and max length.
- After checking all centers, return the substring starting at *start* with length *maxLen*.

Time Complexity: $O(n^2)$ -> Not Good

Video Link: <https://www.youtube.com/watch?v=uX0-xyPkR2w>

CODE :-

```
class Solution {
public:
    string longestPalindrome(string s) {
        if (s.length() < 1) return "";
        int start = 0, maxLen = 0;

        for (int i = 0; i < s.length(); i++) {
            // Check for odd length (center at i) and even length (center between i and i+1) [cite: 17]
            int len1 = expandAroundCenter(s, i, i);
```

```

    int len2 = expandAroundCenter(s, i, i + 1);
    int len = max(len1, len2);

    if (len > maxLen) { // Update start and max length [cite: 19]
        maxLen = len;
        start = i - (len - 1) / 2;
    }
}
return s.substr(start, maxLen); // Return the final substring [cite: 20]
}

```

private:

```

int expandAroundCenter(string s, int left, int right) {
    // Expand outward while characters match and are within bounds [cite: 18]
    while (left >= 0 && right < s.length() && s[left] == s[right]) {
        left--;
        right++;
    }
    return right - left - 1;
}
};

```

OUTPUT :-

The screenshot shows a competitive coding interface. On the left, the problem description for '5. Longest Palindromic Substring' is displayed, including examples and a hint. On the right, the 'Code' editor shows the C++ solution, and the 'Test Result' panel shows that the solution is 'Accepted' with a runtime of 0 ms. The test cases are 'Case 1' and 'Case 2', both of which are passed. The input for Case 1 is 's = "babad"' and the output is '"bab"'. The expected output is also '"bab"'. The input for Case 2 is 's = "cbabd"' and the output is '"bb"'. The expected output is also '"bb"'. The solution code is as follows:

```

int len2 = expandAroundCenter(s, i, i + 1);
int len = max(len1, len2);

if (len > maxLen) { // Update start and max length [cite: 19]
    maxLen = len;
    start = i - (len - 1) / 2;
}
}
return s.substr(start, maxLen); // Return the final substring [cite: 20]
}

```

2. Implement the KMP pattern matching algorithm to search for a pattern in a text.

Given two strings **txt** and **pat**, the task is to return all indices of occurrences of **pat** within **txt**.

Examples:

Input: txt = "abcaab", pat = "ab"

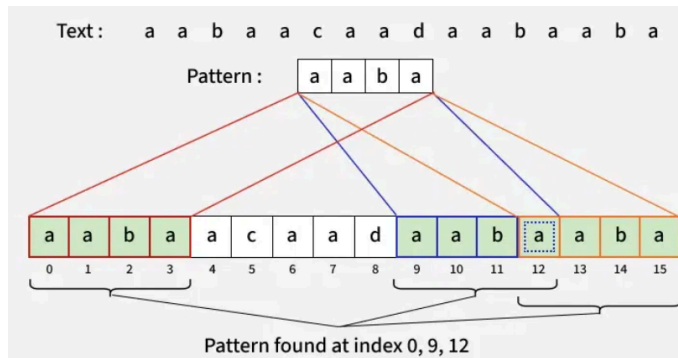
Output: [0, 3]

Explanation: The string "ab" occurs twice in txt, first occurrence starts from index 0 and second from index 3.

Input: txt= "aabaacaadaabaaba", pat = "aaba"

Output: [0, 9, 12]

Explanation:



[Naïve Approach]: We start at every index in the text and compare it with the first character of the pattern, if they match we move to the next character in both text and pattern.

- If there is a mismatch, we start the same process for the next index of the text.

Time Complexity : $O(n^2)$

[Better Approach]: KMP Algorithm

The basic idea behind KMP's algorithm is: whenever we detect a mismatch (after some matches), we already know some of the characters in the text of the next window. We take advantage of this information to avoid matching the characters that we know will anyway match.

Matching Overview

txt = "AAAAABAAABA"

pat = "AAAA"

We compare first window of **txt** with **pat**

txt = "AAAAABAAABA"

pat = "AAAA" [Initial position]

We find a match. This is same as [Naive String Matching](#).

In the next step, we compare next window of **txt** with **pat**.

txt = "AAAAABAAABA"

pat = "AAAA" [Pattern shifted one position]

This is where KMP does optimization over Naive. In this second window, we only compare fourth A of pattern with fourth character of current window of text to decide whether current window matches or not. Since we know first three characters will anyway match, we skipped matching first three characters.

In KMP Algorithm,

- We preprocess the pattern and build LPS array for it. The size of this array is same as pattern length.
- LPS is the **Longest Proper Prefix** which is also a **Suffix**. A proper prefix is a prefix that doesn't include **whole** string. For example, prefixes of "abc" are "", "a", "ab" and "abc" but proper prefixes are "", "a" and "ab" only. Suffixes of the string are "", "c", "bc", and "abc".
- Each value, **lps[i]** is the length of longest proper prefix of **pat[0..i]** which is also a suffix of **pat[0..i]**.

Algorithm for Construction of LPS Array

lps[0] is always **0** since a string of length one has no non-empty proper prefix. We store the value of the previous LPS in a variable **len**, initialized to 0. As we traverse the pattern, we compare the current character at index **i**, with the character at index **len**.

Case 1 - pat[i] = pat[len]: this means that we can simply extend the LPS at the previous index, so increment len by 1 and store its value at lps[i].

Case 2 - pat[i] != pat[len] and len = 0: it means that there were no matching characters earlier and the current characters are also not matching, so lps[i] = 0.

Case 3 - pat[i] != pat[len] and len > 0: it means that we can't extend the LPS at index **i-1**. However, there may be a smaller prefix that matches the suffix ending at i. To find this, we look for a smaller suffix of **pat[i-len...i-1]** that is also a proper prefix of pat. We then attempt to match **pat[i]** with the next character of this prefix. If there is a match, **pat[i] = length of that matching prefix**. Since lps[i-1] equals len, we know that pat[0...len-1] is the same as pat[i-len...i-1]. Thus, rather than searching through pat[i-len...i-1], we can use lps[len - 1] to update len, since that part of the pattern has already been matched.

Simulation for construction of LPS array:

<https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/>

Video: <https://www.youtube.com/watch?v=ynv7bbcSLKE>

Implementation of KMP Algorithm

We initialize two pointers, one for the text string and another for the pattern. When the characters at both pointers match, we **increment** both pointers and continue the

comparison. If they do not match, we reset the pattern pointer to the **last value** from the LPS array, because that portion of the pattern has already been **matched** with the text string. Similarly, if we have traversed the entire pattern string, we add the **starting** index of occurrence of pattern in text, to the result and continue the search from the lps value of **last** element of the pattern.

Let's say we are at position i in the text string and position j in the pattern string when a mismatch occurs:

- At this point, we know that **pat[0..j-1]** has already matched with **txt[i-j..i-1]**.
- The value of **lps[j-1]** represents the length of the longest **proper prefix** of the substring **pat[0..j-1]** that is also a **suffix** of the same substring.
- From these two observations, we can conclude that there's no need to recheck the characters in **pat[0..lps[j-1]]**. Instead, we can directly resume our search from **lps[j-1]**.

CODE :-

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

void computeLPS(string pat, vector<int>& lps) {
    int len = 0, i = 1;
    lps[0] = 0;
    while (i < pat.length()) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) len = lps[len - 1]; // Case 3: Mismatch, use previous LPS [cite: 49]
            else { lps[i] = 0; i++; } // Case 2: No match [cite: 43]
        }
    }
}

void KMP(string txt, string pat) {
    int n = txt.length(), m = pat.length();
    vector<int> lps(m);
    computeLPS(pat, lps);
    int i = 0, j = 0;
    while (i < n) {
        if (pat[j] == txt[i]) { i++; j++; }
        if (j == m) {
```

```

        cout << "Pattern found at index " << i - j << endl;
        j = lps[j - 1];
    } else if (i < n && pat[j] != txt[i]) {
        if (j != 0) j = lps[j - 1];
        else i++;
    }
}
}

int main() {
    string txt = "aabaacaadaabaaba", pat = "aaba";
    KMP(txt, pat);
    return 0;
}

```

OUTPUT :-

Output

```

Pattern found at index 0
Pattern found at index 9
Pattern found at index 12

```

=== Code Execution Successful ===

3. Count all palindromic substrings of a string efficiently.

Given a string *s*, return *the number of palindromic substrings in it*. A string is a **palindrome** when it reads the same backward as forward. A **substring** is a contiguous sequence of characters within the string.

Example 1:

Input: s = "abc"

Output: 3

Explanation: Three palindromic strings: "a", "b", "c".

Example 2:

Input: s = "aaa"

Output: 6

Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa"

Video : <https://www.youtube.com/watch?v=FsmwSuQuGxE>

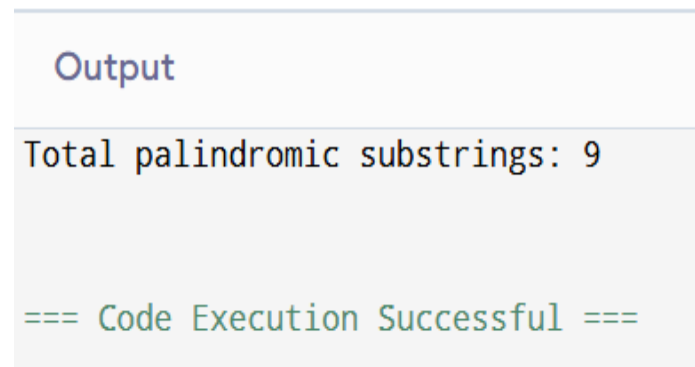
CODE :-

```
#include <iostream>
#include <string>
using namespace std;

int countPalindromes(string s) {
    int n = s.length(), count = 0;
    for (int i = 0; i < n; i++) {
        int l = i, r = i;
        while (l >= 0 && r < n && s[l] == s[r]) {
            count++; l--; r++;
        }
        l = i, r = i + 1;
        while (l >= 0 && r < n && s[l] == s[r]) {
            count++; l--; r++;
        }
    }
    return count;
}

int main() {
    string s = "aaa";
    cout << "Total palindromic substrings: " << countPalindromes(s) << endl;
    return 0;
}
```

OUTPUT :-



```
Output
Total palindromic substrings: 9
=== Code Execution Successful ===
```

4. Find all occurrences of a pattern in a text using the Rabin-Karp algorithm.

Given two strings **text** (the text) and **pattern** (the pattern), consisting of lowercase English alphabets, find all 0-based starting indices where pattern occurs as a substring in text.

Input: text = "geeksforgeeks", pattern = "geeks"

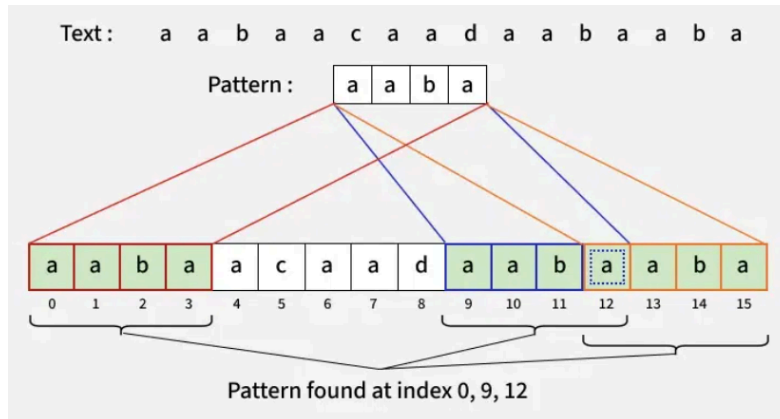
Output: [0, 8]

Explanation: The string "geeks" occurs at index 0 and 8 in text.

Input: text = "aabaacaadaabaaba", pattern = "aaba"

Output: [0, 9, 12]

Explanation:



How is Hash Value calculated in Rabin-Karp?

The hash value is calculated using a **rolling hash function**, which allows you to update the hash value for a new substring by efficiently removing the contribution of the old character and adding the contribution of the new character. This makes it possible to slide the pattern over the **text** and calculate the hash value for each substring without recalculating the entire hash from scratch.

For a string s of length m , the simple hash is

$$\text{hash}(s) = (s[0] \times d^{(m-1)} + s[1] \times d^{(m-2)} + \dots + s[m-1] \times d^0) \% q$$

Here,

$s[i]$ represents the ASCII value of the character ('a' = 97, 'b' = 98, ..., 'z' = 122)

d is the size of the input alphabet (commonly 256 for ASCII characters)

q is a prime number used as modulus (commonly 101, 1000000007, etc.)

Using modulo helps prevent integer overflow and reduces hash collisions.

How do we compute rolling hash?

So we know hash values $s[i-1, j-1]$ and now we need to compute for $s[i..j]$.

Previous value of hash

The hash value of substring $s[i-1 \dots j-1]$ is: $(s[i-1] * d^{(m-1)} + s[i] * d^{(m-2)} + \dots + s[j-1] * d^0) \bmod q$

New value of hash

The hash value of the next substring $s[i \dots j]$ is: $(s[i] * d^{(m-1)} + s[i+1] * d^{(m-2)} + \dots + s[j] * d^0) \bmod q$

From the above expressions, it is clear that we can get new value from old value by removing the term $s[i-1] * d^{(m-1)}$ and adding the term $s[j] * d^0$

$$hash(i, j) = (d * (hash(i-1, j-1) - s[i-1] * d^{(m-1)}) + s[j]) \bmod q$$

Here's how the hash value is typically calculated in Rabin-Karp:

- Choose a base d for the hash function (commonly 31 for lowercase characters or 256 for ASCII). Also select a large prime number q as the modulus to prevent overflow and reduce hash collisions.
- Precompute the value of the term $d^{(m-1)}$ as we are going to repeatedly multiply with this term in rolling hash calculations as explained above.
- Calculate the hash for the pattern and the first substring of text matching. Let the pattern hash function be p and text hash function be t . We use the formula $(s[0] * d^{(m-1)} + s[1] * d^{(m-2)} + \dots + s[m-1] * d^0) \bmod q$ for the first computations.
 - Update the hash by removing the leftmost char and adding the next one using $hash(i, j) = ((hash(i-1, j-1) - s[i-1] * d^{(m-1)}) \bmod q + s[j]) \bmod q$
 - If the current text hash equals the pattern hash, verify character by character. This is needed because different substrings can have the same hash

Time Complexity:

- Average case: $O(n + m)$ - Computing the hash of the pattern and the first substring of text takes $O(m)$, and sliding the window over the text with hash comparisons takes $O(n)$ on average. Character-by-character verification is rarely needed.
- Worst case: $O(n \times m)$ - If many hash collisions occur, each substring may require a full character comparison.

CODES :-

```
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
using namespace std;

void rabinKarp(string txt, string pat) {
    int n = txt.length(), m = pat.length();
    int d = 256;
    int q = 101;
    int h = 1, p = 0, t = 0;

    // Precompute h = d^(m-1) % q [cite: 72]
    for (int i = 0; i < m - 1; i++) h = (h * d) % q;

    // Initial hash values for pattern and first window [cite: 75]
```

```
for (int i = 0; i < m; i++) {
    p = (d * p + pat[i]) % q;
    t = (d * t + txt[i]) % q;
}

for (int i = 0; i <= n - m; i++) {
    if (p == t) { // If hashes match, verify characters [cite: 77]
        bool match = true;
        for (int j = 0; j < m; j++) {
            if (txt[i + j] != pat[j]) { match = false; break; }
        }
        if (match) cout << "Pattern found at index " << i << endl;
    }
    if (i < n - m) {
        t = (d * (t - txt[i] * h) + txt[i + m]) % q;
        if (t < 0) t = (t + q);
    }
}
}

int main() {
    string txt = "geeksforgeeks", pat = "geeks";
    rabinKarp(txt, pat); // Output: 0, 8 [cite: 63]
    return 0;
}
```

OUTPUT :-

Output

Pattern found at index 0

Pattern found at index 8

=== Code Execution Successful ===

5. Find the smallest window in a string that contains all characters of another string.

CODE :-

```
#include <iostream>
#include <vector>
#include <string>
#include <climits>
using namespace std;

string smallestWindow(string s, string p) {
    if (s.length() < p.length()) return "";
    vector<int> target(256, 0), window(256, 0);
    for (char c : p) target[c]++;

    int count = 0, start = 0, minLen = INT_MAX, startIndex = -1;
    for (int j = 0; j < s.length(); j++) {
        window[s[j]]++;
        if (target[s[j]] != 0 && window[s[j]] <= target[s[j]]) count++;

        if (count == p.length()) {
            // Minimize the window from the left
            while (window[s[start]] > target[s[start]] || target[s[start]] == 0) {
                window[s[start]]--;
                start++;
            }
            if (j - start + 1 < minLen) {
                minLen = j - start + 1;
                startIndex = start;
            }
        }
    }
    return (startIndex == -1) ? "" : s.substr(startIndex, minLen);
}

int main() {
    string s = "ADOBECODEBANC", p = "ABC";
    cout << "Smallest window: " << smallestWindow(s, p) << endl;
    return 0;
}
```

OUTPUT :-

Output

Smallest window: BANC

=== Code Execution Successful ===

Conclusion: (Write your conclusion here)

Through this experiment, we gained a comprehensive understanding of advanced string-matching algorithms like KMP and Rabin-Karp, focusing on how pre-processing techniques and rolling hashes optimize search complexity. We also mastered the application of expansion and sliding window strategies to solve complex problems involving palindromes and character-based substring searches. Ultimately, the lab demonstrated how these algorithmic optimizations provide significant performance improvements over naive approaches when handling large-scale textual data.

Post Lab Questions:

1. Compute time and space complexity of all your algorithms

Post LAB

PAGE No.	
DATE	/ /

1) Longest Palindromic substring

Time complexity :- $O(n^2)$

there are $2n-1$ possible centers for a palindrome in a string of length n . for each center, we expand outwards which takes $O(n)$ time in worst case

Space complexity :- $O(1)$

the algo uses few integer variables to track the substring requiring no extra data structures

2) KMP Pattern matching algorithm

Time complexity :- $O(n+m)$ Preprocessing the pattern to create the LPS array takes $O(m)$ time, and the actual searching process through the text takes $O(n)$ time without backtracking

Space complexity :- $O(m)$

we require an auxiliary array (LPS array) of size m to store longest prefix-suffix value

3) Count all Palindromic substring

Time complexity :- $O(n^2)$ similar to the longest palindrome algorithm, we visit each of the $2n-1$ potential center and expand outwards to count matches leading to quadratic time

Space complexity :- $O(1)$
we only maintain a counter variable and few pointers to track the current expansion, consuming constant extra space

4) Rabin Karp algorithm

Time complexity :- avg :- $O(n+m)$ worst :- $O(n \times m)$
On average, the rolling hash allows $O(1)$ comparisons per window. However, in the worst case it can degrade to $O(n \times m)$

Space complexity :- $O(1)$ The algorithm only stores the hash values for the pattern and the current test window, which occupy constant space

5) Smallest window with all characters

Time complexity :- $O(n)$

we use two pointers to create a sliding window
Each character in the string is ~~not~~ visited at most twice

Space complexity :- $O(1)$

we used fixed size array to store character counts
since the array size does not grow with the input size n , it is considered constant space